Getting started with InstructLab for generative AI model tuning

June 12, 2024 | Cedric Clyburn

Related topics: Artificial intelligence, Data science, Developer tools, Linux, Open source

Related products: Developer Tools, Podman Desktop, Red Hat Enterprise Linux AI

Table of contents:

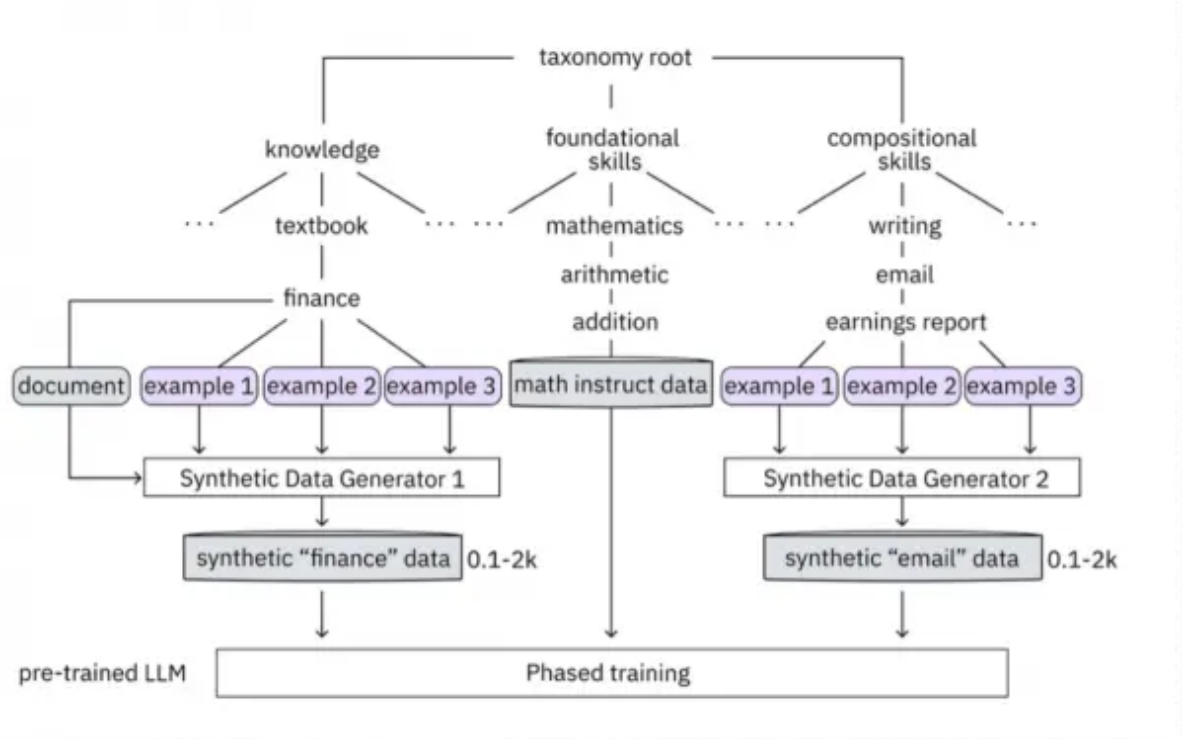
Recent advances in large language models (LLMs) have unlocked exciting new natural language processing and generation capabilities. However, tuning these massive models to specific domains or tasks can be prohibitively expensive and technically complex, often requiring immense GPU computation, extensive training time, and deep expertise.
The InstructLab open source project aims to democratize LLM tuning by enabling community-driven, low-resource model refinement. Let's learn how to get started with InstructLab on your local system, and go through the process to contribute to an existing LLM and enhance a model with new data.

What is InstructLab?

Initiated by IBM and Red Hat, InstructLab provides tools to enhance LLMs with additional knowledge and skills using a novel approach called LAB (Large-Scale Alignment for ChatBots). InstructLab empowers subject matter experts to iteratively refine foundation models for targeted use cases, without needing deep machine learning expertise or huge training budgets. What makes InstructLab unique comes from several factors:

- A collaborative, taxonomy-driven knowledge curation workflow
- Efficient synthetic data generation to maximize the data from human-provided examples

- The instruction training using LAB that delivers big gains from small datasets



Getting started with InstructLab

We'll walk through getting started with InstructLab, from installation to contributing skills & knowledge, generating synthetic data, tuning the model, and testing the results. By the end, you'll see how InstructLab makes powerful LLM tuning accessible to developers and data scientists of all skill levels.

Installing the InstructLab CLI

InstructLab provides a command-line interface (CLI) called **ilab** that handles the main tuning workflow. Currently, it supports Linux systems and Apple Silicon Macs (M1/M2/M3), as well as Windows with WSL2 (check out this guide). In addition, you'll need Python 3.9+ (<3.12 for PyTorch JIT), a C++ compiler, and about 60GB of free disk space, more information is in the project's README.

Now, let's go ahead and create a new directory for your InstructLab project and set up a Python virtual environment so that the InstructLab

components installed are just available in this environment and not system-wide.

```
$ mkdir instructlab && cd instructlab
$ python3 -m venv --upgrade-deps venv
$ source venv/bin/activate
$ pip cache remove llama_cpp_python
```

While there are several ways to install InstructLab, the easiest is to use pip to install the CLI (or specific release of InstructLab, and for this example, we'll be using 0.17.1).

```
$ pip install 'instructlab[mps]==0.17.1'
```

This will compile the native dependencies and download the required Python packages. It may take a few minutes to finish, but when ready, let's validate the installation.

```
$ ilab --version
ilab, version 0.17.1
```

You should see the ilab version printed, e.g. **ilab, version 0.17.1**.

Initializing InstructLab and a Taxonomy project

With ilab installed, we can initialize our tuning environment with the **ilab config init** command. This will download the Taxonomy repository (which contains the structured task definitions and community-provided knowledge) to our local machine with a configuration file (after selecting **Enter** and **Yes** for project defaults).

```
$ ilab config init
Welcome to InstructLab CLI. This guide will help you to
setup your environment.
Please provide the following values to initiate the
environment [press Enter for defaults]:
Path to taxonomy repo [taxonomy]:
`taxonomy` seems to not exist or is empty. Should I clone
https://github.com/instructlab/taxonomy.git for you? [y/N]:
Yes
```

```
Cloning https://github.com/instructlab/taxonomy.git...
Generating `config.yaml` in the current directory...
Initialization completed successfully, you're ready to
start using `ilab`. Enjoy!
```

Let's also examine the configuration file, which defines the foundational model we'll be training and includes defaults such as parameters for training and serving.

```
$ cat config.yaml
chat:
 context: default
 greedy_mode: false
 logs_dir: data/chatlogs
 max_tokens: null
 model: models/merlinite-7b-lab-Q4_K_M.gguf
 session: null
 vi_mode: false
 visible_overflow: true
general:
 log_level: INFO
generate:
 chunk_word_count: 1000
 model: models/merlinite-7b-lab-Q4_K_M.gguf
 num_cpus: 10
 num_instructions: 100
…
```

Downloading, serving, and testing a model with InstructLab

InstructLab can enhance open-source LLMs like Mistral and Llama, and now, the Granite set of foundation models in partnership with IBM and Red Hat. To get started, download a compact pre-trained & quantized model with the **ilab model download** command.

```
$ ilab model download
Downloading model from instructlab/merlinite-7b-lab-
GGUF@main to models...
```

```
Downloading 'merlinite-7b-lab-Q4_K_M.gguf' to
'models/.huggingface/download/merlinite-7b-lab-
Q4_K_M.gguf.9ca044d727db34750e1aeb04e3b18c3cf4a8c064a9ac96c
f00448c506631d16c.incomplete'
INFO 2024-06-11 23:21:23,255 file_download.py:1877
Downloading 'merlinite-7b-lab-Q4_K_M.gguf' to
'models/.huggingface/download/merlinite-7b-lab-
Q4_K_M.gguf.9ca044d727db34750e1aeb04e3b18c3cf4a8c064a9ac96c
f00448c506631d16c.incomplete'
merlinite-7b-lab-Q4_K_M.gguf:   2%|▌        | 105M/4.37G
[01:23<57:18, 1.24MB/s]
```

This Merlinite (derivative of Mistral) model, in particular, is trained with 7 billion parameters and is roughly 4GB in size, so it may take a few minutes on a fast connection. You can also download a specific model version or an entire Hugging Face repo. Now, let's serve the model to be inferenced from our local machine using **ilab model serve**.

```
$ ilab model serve
INFO 2024-06-11 23:27:21,994 lab.py:340 Using model
'models/merlinite-7b-lab-Q4_K_M.gguf' with -1 gpu-layers
and 4096 max context size.
INFO 2024-06-11 23:27:40,984 server.py:206 Starting server
process, press CTRL+C to shutdown server...
INFO 2024-06-11 23:27:40,984 server.py:207 After
application startup complete see http://127.0.0.1:8000/docs
for API.
```

Now, in a new terminal environment, enter your virtual environment again (**source venv/bin/activate**), navigate to your InstructLab directory, and run **ilab model chat** to enter a simple interface for conversing with the LLM.

```
$ ilab model chat
╭──────────────────────────────────────────────────────────────────────────────╮
│ Welcome to InstructLab Chat w/ MODELS/MERLINITE-7B-LAB-
Q4_K_M.GGUF (type /h for help)
│
╰──────────────────────────────────────────────────────────────────────────────╯
>>> What languages are spoken in Canada?
```

```
╭──────────────────────────────────────────── models/
merlinite-7b-lab-Q4_K_M.gguf
│ Canadian society is multilingual, with English and French
being the two official languages recognized at the federal
level.
```

You'll notice that this format may be different than your output! In this interactive terminal for conversing and testing out the model, type **/h** to learn about all of the different commands available, such as **/p** for plain text (without the ASCII), **/n** for a new chat context, and **/q** to quit the chat.

Model alignment and training with InstructLab

Now that we have a foundational model downloaded, served, and tested, let's dive into the exciting part: aligning and training the model with new knowledge and skills! InstructLab provides a structured approach to curating knowledge and generating synthetic data for efficient model tuning.

As an example, I'm feeling lucky, so let's ask this model about the largest Las Vegas jackpot wins through the **ilab model chat** interface (that you may still have open).

```
$ ilab model chat
>>> When and how much was the largest Las Vegas jackpot?
╭──────────────────── models/merlinite-7b-lab-Q4_K_M.gguf
────
The largest jackpot in Las Vegas history was $38,450,000,
which was won by a lucky player on October 27, 2018. This
massive payout occurred at the MGM Grand casino, making it
one of the most significant wins ever recorded in the
city's gambling scene.
```

That's fantastic! However, upon reviewing the actual world record for largest slot machine win, there are some big differences, such as the date (October 27, 2018 should be March 21, 2003) and payout ($38,450,000 should be $39,713,982). Large Language Models, while impressive in their ability to conversate and recall training information, sometimes tend to hallucinate or simply aren't aware of specific details due to their large

training set of data. Let's learn how to contribute the correct information to this model using InstructLab!

Adding knowledge and skills to an LLM

Navigate to the taxonomy directory that was cloned during the initialization step. Here, you'll find two main subdirectories: knowledge and skills. As the names suggest, knowledge refers to factual information you want to add to the model, while skills involve teaching the model to perform specific tasks or follow certain formats. Feel free to leave the chat interface with the **/q** command, and let's navigate to the Taxonomy directory to understand how to contribute to the model with InstructLab:

```
$ cd taxonomy && ls
CODE_OF_CONDUCT.md    SECURITY.md
CONTRIBUTING.md        compositional_skills
CONTRIBUTOR_ROLES.md    docs
LICENSE            foundational_skills
MAINTAINERS.md        governance.md
Makefile        knowledge
README.md        schema
```

Let's say we want to add knowledge about the biggest Las Vegas jackpots, which would be considered an addition of knowledge to the model. First, let's view some of the default and example directories included in the project already.

```
$ cd knowledge
Humanities        insects
README.md        knots
STEM            knowledge_domains.md
animals            laws
archeology        literature
```

Inside this directory, we're going to create a new subfolder and qna.yaml (questions & answers) file to hold example question-answer pairs related to the jackpots, just like the one below.

```
task_description: 'Teach the model about record breaking
```

```
Las Vegas jackpots
created_by: cedricclyburn
domain: trivia
seed_examples:
 - question: When and where was the biggest Las Vegas slot
machine jackpot won?
   answer: |
     The biggest Las Vegas slot jackpot of $39,713,982.25
was won on March 21, 2003 on a
     Megabucks machine at the Excalibur Hotel-Casino in Las
Vegas, Nevada, USA.
 - question: How much money did the winner put in the slot
machine before hitting the record jackpot?
   answer: The winner put $100 into the Megabucks machine
before winning the $39,713,982.25 jackpot.
 - question: What was the occupation and hometown of the
winner of the largest Vegas slot jackpot?
   answer: |
     The winner of the $39,713,982.25 Megabucks jackpot in
2003 was a 25-year-old software
     engineer from Los Angeles.
 - question: What is the exact amount of the largest slot
machine payout in Las Vegas?
   answer: The largest slot machine payout in Las Vegas is
$39,713,982.25.
 - question: How much was the record-breaking Las Vegas
jackpot worth in British pounds?
   answer: The $39,713,982.25 jackpot won in Las Vegas on
March 21, 2003 was equivalent to £25,361,761.
document:
 repo: https://github.com/cedricclyburn/las-vegas-gambling-
knowledge.git
 commit: 2f3b881
 patterns:
   - biggest_vegas_jackpots.md
```

You can begin to understand how the file is structured, from the metadata to the 5-10 Q&A pairs that can be included (although there's an approximate limit of 2,300 words for training efficiency and quality). There is also a link to a public repository for additional data points from which InstructLab will generate additional question and answer pairs. This

InstructLab will generate additional question-and-answer pairs. This additional data will be used to generate synthetic question-answer pairs during the next step.

If you'd like to use this same qna.yaml file, feel free to download it locally using this command from the **taxonomy/knowledge/ directory**, which will also create a new directory structure under the knowledge directory for trivia/vegas.

```
$ mkdir -p trivia/vegas && curl -sSL https://
raw.githubusercontent.com/cedricclyburn/vegas-taxonomy/
main/knowledge/trivia/vegas/qna.yaml -o trivia/vegas/
qna.yaml
```

Generating synthetic data for model training

So we've added some task-specific knowledge—now what? The next step is to use InstructLab's synthetic data generation pipeline to create a large training set from your examples. The key insight behind InstructLab's LAB method is that we can use the base model itself to massively expand a small set of human-provided examples. By prompting the model to generate completions conditioned on your examples, we can produce a synthetic dataset that's much larger and more diverse than what you could feasibly write by hand.

First, let's list, compare, and validate the new data compared to the base taxonomy repository using the **ilab taxonomy diff** command, from your base InstructLab directory (where the config.yaml is located).

```
$ ls
config.yaml    data        models        taxonomy
$ ilab taxonomy diff
knowledge/trivia/vegas/qna.yaml
Taxonomy in /taxonomy/ is valid :)
```

Now, we can run the **ilab data generate** command to begin generating synthetic data (by default, 100 data points). Remember, we still need to be serving the model with **ilab model serve** in another terminal instance.

```
$ ilab data generate
Q> How much money did the winner of the biggest jackpot
win?
I>
A> The winner won a record-breaking $39,713,982.25 (US
Dollars) on March 21, 2003. This amount is equivalent to
approximately £25,361,761 in British pounds.
2%|▌                                  | 2/100 [00:25<08:21,
5.12s/it]INFO 2024-06-12 00:20:58,889 generate_data.py:468
Selected taxonomy path knowledge->trivia->vegas
```

During this process, we can visualize the example queries and answers, light preprocessing, and prompts being fed to the base model to generate a large number of candidate completions. The generated completions are filtered and post-processed to remove low-quality or irrelevant outputs. This is a critical step, as the model can sometimes generate nonsensical or factually incorrect responses.

On a typical laptop CPU, synthetic data generation can take anywhere from a few minutes to a few hours, depending on the number of examples and generation parameters. Using a GPU will significantly speed things up. The end result is a set of JSONL files in the specified output directory, with a train/validation/test split. Each line contains an example with input and target completion fields, and feel free to vim to understand how things are working under the hood.

```
$ cd generated && ls
discarded_merlinite-7b-lab-Q4_K_M_2024-06-12T00_20_33.log
generated_merlinite-7b-lab-Q4_K_M_2024-05-29T19_28_12.json
test_merlinite-7b-lab-Q4_K_M_2024-06-12T00_20_33.jsonl
train_merlinite-7b-lab-Q4_K_M_2024-05-29T19_28_12.jsonl
```

Model training (alignment) with InstructLab

Once the synthetic data generation is complete, it's time to actually tune the model on the synthetic data with **ilab model train**.

```
$ ilab model train
[INFO] Loading
```

```
[INFO] Loading
Fetching 11 files: 100%|
└└└
```

```
special_tokens_map.json
```

Testing the new model and next steps

Finally, to interact with the aligned model and observe the impact of the added knowledge or skills, let's serve the model by pointing to the new model path with **ilab model serve**.

```
$ ilab model serve --model-path instructlab-merlinite-7b-
lab-trained/instructlab-merlinite-7b-lab-Q4_K_M.gguf
INFO 2024-06-12 01:07:12,714 lab.py:340 Using model
'instructlab-merlinite-7b-lab-trained/instructlab-
merlinite-7b-lab-Q4_K_M.gguf' with -1 gpu-layers and 4096
max context size.
INFO 2024-06-12 01:07:30,935 server.py:206 Starting server
process, press CTRL+C to shutdown server...
```

With the model served, we can switch over to our other terminal instance and use **ilab model chat** to converse with the model and verify the new knowledge, also pointing to the new quantized model.

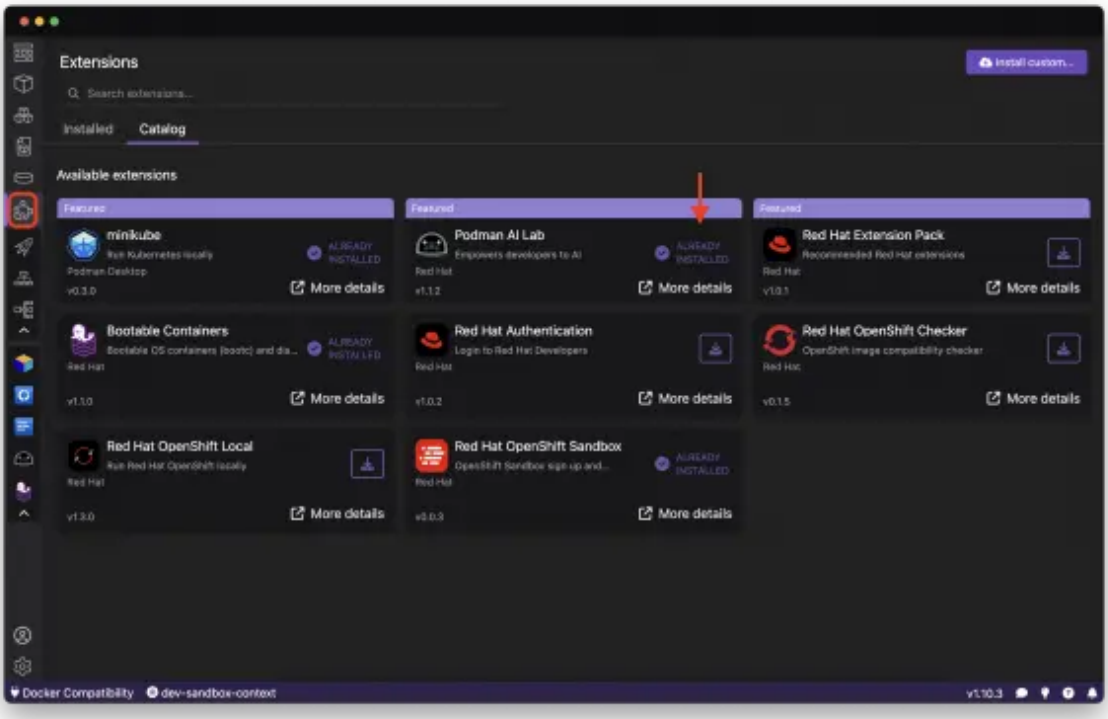
```
$ ilab model chat --model instructlab-merlinite-7b-lab-
trained/instructlab-merlinite-7b-lab-Q4_K_M.gguf
╭──────── instructlab-merlinite-7b-lab-trained/instructlab-
merlinite-7b-lab-Q4_K_M.gguf
>>> How much and when was the largest Las Vegas jackpot?
The largest Las Vegas jackpot of $39,713,982.25 (US
Dollars) was won by a 25-year-old software engineer from
Los Angeles on March 21, 2003.
```

Fantastic! It recalls the exact information we provided in the taxonomy repository. We've successfully performed model alignment on consumer-grade hardware and tailored this LLM for our specific use case.

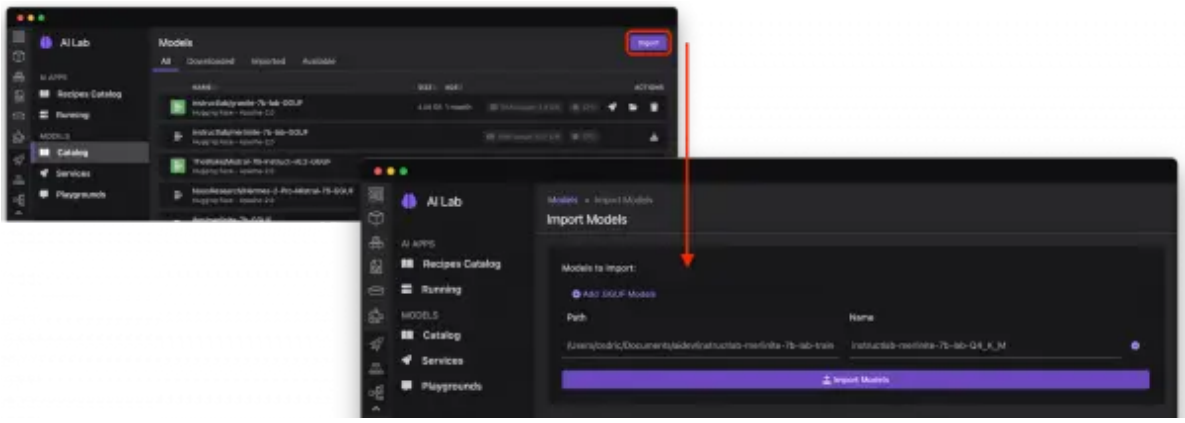
Building Generative AI-enabled applications with Podman AI Lab

You may be interested in taking this model, in quantized .gguf format, and using it to build AI-enabled applications. Fortunately, the Podman AI Lab

extension for Podman Desktop (the graphical open-source tool for working with containers and Kubernetes) helps with just that. After downloading and setting up Podman Desktop from podman-desktop.io, navigate to the **Extensions** in the left-sidebar to easily install the **AI Lab** extension.

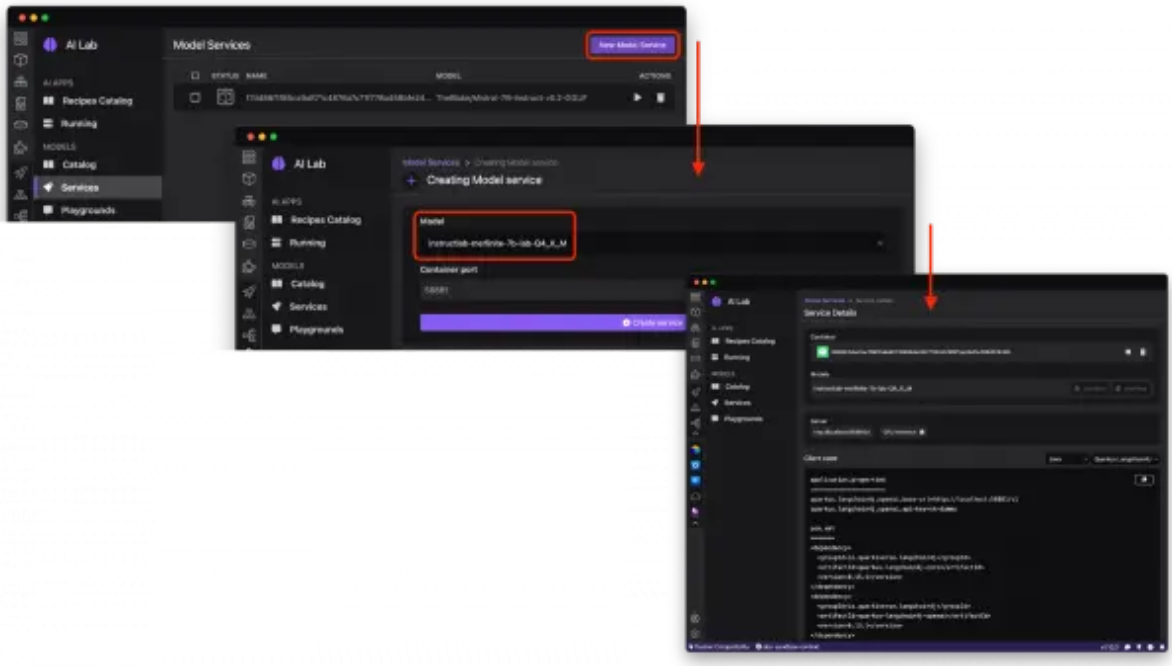


Now, from the **Catalog** of models, you can import a new model from your local machine (for example, the new InstructLab aligned model, or one from HuggingFace).



Finally, you can create a new model **Service** (or Playground for model experimentation) to seamlessly integrate the power of generative AI into your application, with information about the model's API endpoint (which

is OpenAI API compatible) and sample code to get you started in your favorite language or framework.



Conclusion

Congratulations! You have successfully installed InstructLab, downloaded and served a foundational model, added new knowledge, generated synthetic data, trained the model, and tested the aligned model with the added knowledge. InstructLab makes it possible for developers and domain experts to collaborate on enhancing large language models without requiring extensive machine learning expertise or massive computational resources.

To dive deeper, check out the InstructLab organization on Github and join the community Slack to collaborate with other contributors. As the InstructLab ecosystem grows, it has the potential to continuously refine open LLMs to be more knowledgeable, capable, and aligned with human values. We're excited to see what you'll create!

Last updated: November 3, 2024

Disclaimer: Please note the content in this blog post has not been thoroughly

reviewed by the Red Hat Developer editorial team. Any opinions expressed in this post are the author's own and do not necessarily reflect the policies or positions of Red Hat.

Related Posts

InstructLab: Advancing generative AI through open source

Apply generative AI to app modernization with Konveyor AI

How to integrate Quarkus applications with OpenShift AI

Building machine learning models in the cloud

Model training in Red Hat OpenShift AI

Experiment and test AI models with Podman AI Lab

Recent Posts

Manage AI context with the Lola package manager

Agent-driven attestation: How Keylime's push model rethinks remote integrity verification

Harness engineering: Structured workflows for AI-assisted development

Running Karpathy's autoresearch on Red Hat OpenShift AI: 198 experiments, zero intervention

Distributed tracing for agentic workflows with OpenTelemetry

Platforms

Red Hat AI

Red Hat Enterprise Linux

Red Hat OpenShift

Red Hat Ansible Automation Platform

See all products

Build

Developer Sandbox

Developer tools

Interactive tutorials

API catalog

Quicklinks

Learning resources

E-books

Cheat sheets

Blog

Events

Newsletter

Communicate

About us

Contact sales

Find a partner

Report a website issue

Site status dashboard

Report a security problem

RED HAT DEVELOPER

Build here. Go anywhere.

We serve the builders. The problem solvers who create careers with code.

Join us if you're a developer, software engineer, web designer, front-end designer, UX designer, computer scientist, architect, tester, product manager, project manager or team lead.

Sign me up →

About Red Hat

Jobs

Events

Locations

Contact Red Hat

Red Hat Blog

Inclusion at Red Hat

Cool Stuff Store

Red Hat Summit

© 2026 Red Hat

Privacy statement

Terms of use

All policies and guidelines

Digital accessibility

Cookie preferences